Amauri Hopewell

Southern New Hampshire University

CS-499 Computer Science Capstone

October 18, 2026

**Professional Self-Assessment**

When I first completed the projects in this portfolio, I was mainly concerned with making each program work and meeting the requirements for the course. Revisiting the code for CS-499 made me evaluate it differently. Some solutions that were adequate for an assignment had weaknesses in maintainability, performance, or security once I imagined someone else using or extending them. I chose three projects to show what I learned from that second look: the Contact service from CS-320 (software design and engineering), the ABCU course catalog from CS-300 (algorithms and data structures), and AnimalShelter for Grazioso Salvare from CS-340 (databases).

My concentration is software engineering. This self-assessment introduces the enhanced projects and explains how they support the five CS-499 program outcomes (Southern New Hampshire University [SNHU], n.d.). The ePortfolio is available at https://amaurihopewell.github.io/CS-499/. I kept the original coursework on each repository's main branch and placed the enhancements on separate milestone branches, so readers can compare the before-and-after versions.

**Coursework, strengths, and employability**

The most important change in my approach has been thinking about what happens after a program passes its initial assignment tests. In CS-320, I learned to preserve and test business rules while changing the structure around them. In CS-300, I learned to compare data structures based

on how they perform the operations a program actually needs. In CS-340, I became more attentive to the risks created by database credentials, unexpected query input, and overly broad write operations. The capstone gave me a chance to apply those lessons to older work rather than just describing them in the abstract.

I am seeking software engineering internships and junior developer or software development engineer in test (SDET) opportunities in Washington, D.C. and Northern Virginia, including on-site, hybrid, and remote positions, before graduation and around May 2027. In my Module Four career research, I found postings asking for Java or Python, Git, SQL, testing experience, and an understanding of HTTP behavior. I can now point a prospective employer to the Contact API and its tests, the course-catalog comparison program, and the AnimalShelter test suite as evidence of those skills. That is much more persuasive than simply naming technologies on a résumé.

An experience outside of school also influences how I approach this work. While processing Amazon returns in Lexington, I learned that an item could look fine at first glance but still cause a problem for the next person handling it. That made me more careful about checking details before passing work along. In software, I see a similar responsibility when I inspect an API response for information it should not reveal, check what a database deletion will affect, or write instructions that someone else must follow to run a project.

**Collaborating in a team environment**

I completed these capstone enhancements individually, so I do not want to suggest that creating Git branches is the same as working on a development team. What the project did teach me was how to make my work easier for other people to review. My Module Two code-review video walks through the original CS-320 Contact service, including its field restrictions, the

discrepancy between its ArrayList implementation and a README describing a HashMap, and a println statement in getContact. Recording the review forced me to explain what the existing code did, why something concerned me, and what I intended to change, rather than assuming a viewer could read my mind.

I also separated each enhancement from the original code in branches named cs499-milestone-two, cs499-milestone-three, and cs499-milestone-four. A reviewer can inspect the original and enhanced versions without reconstructing the changes. In seeking review, I identified specific questions about the Contact authentication responses, the median-first binary search tree and scheduling menu in CS-300, and the unique-index behavior when an existing animal dataset contains duplicate IDs. The milestone grades did not require another code revision; I therefore cannot point to those grades as evidence of a particular fix prompted by feedback. The more concrete lesson was that useful collaboration begins with giving another person enough context to evaluate a decision and challenge it, or, in this case, accept it.

**Communicating with stakeholders**

The AnimalShelter enhancement was useful practice in designing for someone who might not care about the underlying Python code. Grazioso staff need a way to see rescue candidates, not an explanation of a MongoDB query. The dashboard offers Water, Mountain, and Disaster rescue filters, along with a results table and breed chart. Its staff role can read records, while its application-level admin role controls writes. I chose those visible controls to make the underlying database functionality understandable in the context of the rescue organization.

For a developer, instructor, or hiring manager, the audience needs something different: a clear explanation of how to install, run, and evaluate the project. I wrote enhanced-folder READMEs with commands for setup and testing and separate narratives explaining the purpose

and results of each enhancement. The code-review video serves readers who want an overview before cloning a repository, while the code and tests let technical reviewers inspect the details. Learning to present the same work at those different levels has been an important part of the capstone.

**Data structures, algorithms, and design trade-offs**

The CS-300 enhancement uses an ABCU course catalog that originally stored course records in a vector. I added comparison functionality for the vector, a binary search tree (BST), and a hash map using a 129-course input file. The exercise made the design trade-offs more concrete. A hash map is useful for finding a course by identifier, while a tree can support ordered traversal; a vector is simple, but repeated searches may require scanning records. Instead of assuming that one structure was always best, I added timing comparisons and documented how to build the comparison program with the /O2 optimization flag.

I also added a prerequisite-aware schedule in menu option 6 using topological ordering. That required a different way of thinking about the data: finding a course quickly does not itself tell you whether the courses can be taken in a valid order. For the BST, I used median-first insertion from sorted identifiers to avoid producing a long chain during the initial build. This gives a more balanced initial tree for that dataset, although it is not a self-balancing tree for arbitrary later inserts. That limitation helped me see why the context of a design choice matters as much as its theoretical advantages.

**Software engineering, databases, and tools**

For CS-320, I kept the original ContactService validation rules, including a final contact ID, ten-character name limits, a 30-character address limit, and a ten-digit phone number. My enhancement placed a repository layer beneath the service. Tests use an in-memory HashMap,

while the running application uses SQLite so contacts persist across restarts. I also added an HTTP interface using the JDK HttpServer rather than Spring. That choice kept the demonstration relatively small, but it also meant I had to be explicit about request parsing and response behavior. The enhanced project has 37 tests that can be run with mvn test. I now appreciate separation of responsibilities not just as a design principle but as something that makes individual parts easier to test and change.

For CS-340, I consolidated create, read, update, and delete operations in the AnimalShelter module. The enhancement adds a unique index on animal_id, an allow-list for lookup fields, rejection of certain operator keys such as $where, and application-level staff/admin write controls. The MongoDB credentials come from the environment rather than being embedded in the source. The 12 pytest tests use mongomock. If credentials are not supplied, running python dashboard.py opens a demo with seven sample dogs instead of requiring an instructor to configure a MongoDB server merely to see the interface. Taken together, these changes show my growing ability to work across a service, database, test environment, and user interface.

**Security**

Security is one of the clearest differences between the original coursework and my enhanced projects. Earlier assignments focused on whether the program performed the requested operation. Revisiting them made me ask who should be able to perform that operation, what should happen when a request is unauthorized, and whether a failure could expose data. These questions directly support the program's fifth outcome (SNHU, n.d.).

In Contact, the application requires configured staff and admin passwords rather than silently using a default password. Tests distinguish an unauthenticated request (HTTP 401) from an authenticated staff user attempting an admin-only write (HTTP 403). The tested responses use

an empty JSON object, {}, without returning contact details such as the sample phone number. The AuditLog records the actor, action, identifier, and status instead of copying names, phone numbers, or addresses into its entries. These are small decisions, but each prevents an avoidable disclosure or an ambiguous failure.

In AnimalShelter, a live database connection requires an available client and configured credentials; the separate demo mode does not pretend to be a secured production database. Empty delete criteria return zero rather than being passed through as a broad deletion. This matters because the original CS-340 implementation used a many=True deletion path that could affect more records than intended. One remaining limitation is that the dashboard's staff/admin distinction is implemented at the application level while both roles use the same MongoDB connection. A production deployment would require a closer review of database-level privileges and the insert-input validation as well.

**Future Improvements**

The enhancements also show where there are further opportunities for improvements. Creating a unique index is straightforward on clean demonstration data, but an existing AAC dataset with duplicate animal identifiers would need a deliberate cleanup or migration plan. I would also revisit the AnimalShelter insert payload, which does not yet have the same field allow-list as lookups, and test against a populated live database. The dashboard includes a table and a breed chart, but not the map I considered in an early plan. I would rather describe those limits than present the software as more complete than it is.

I learned that finishing a milestone and finishing a product are not necessarily the same thing. My 3-2, 4-2, and 5-2 milestone evaluations did not call for another submission, but there are still design decisions I would revisit with more time or with the help of another developer. In

particular, I would prioritize stronger validation, a safer path for importing imperfect existing data, and more testing of the security boundaries outside the mock environment.

**How the artifacts fit together**

The three artifacts represent software design and engineering, algorithms and data structures, and databases, as required by the capstone (SNHU, n.d.). They also show different sides of problems that come up in real applications. Contact made me think about validation, persistence, authorization, and API behavior. ABCU made me think about performance and how data structures support different operations. AnimalShelter made me apply similar principles where stored records, user permissions, and a working interface all matter. Using a previous project for each area makes the improvements visible against my original work, as proposed in my Module One plan (Hopewell, 2026).

The code-review video and repository organization address the collaborative and reviewable-work aspects of Outcome 1. The video, documentation, dashboard, narratives, and this assessment address Outcome 2 by communicating with different audiences. The course-catalog comparisons and prerequisite scheduling provide the clearest evidence for Outcome 3. The Java, C++, Python, SQLite, MongoDB, JUnit, pytest, and HTTP work support Outcome 4. Finally, authentication checks, safer database operations, and restrained audit logging support Outcome 5. More importantly, the portfolio shows a change in how I approach a project. I don't just want the program to work, but instead I want it to work while making sure another person can understand, test, use, and safely maintain it.

**Where the artifacts are:**

**Software design:**

- Original: https://github.com/AmauriHopewell/CS320.

- Enhanced: https://github.com/AmauriHopewell/CS320/tree/cs499-milestone-two/enhanced.
- From that enhanced folder, mvn test is 37 tests.
- Narrative: CS-499_3-2_Narrative_Software_Design_Amauri_Hopewell.

**Algorithms:**

- Original: https://github.com/AmauriHopewell/CS300.
- Enhanced: https://github.com/AmauriHopewell/CS300/tree/cs499-milestone-three/enhanced.
    - Build with /O2. The enhanced README has the compare command.
- Narrative: CS-499_4-2_Narrative_Algorithms_Amauri_Hopewell.

**Databases:**

- Original: https://github.com/AmauriHopewell/CS340.
- Enhanced: https://github.com/AmauriHopewell/CS340/tree/cs499-milestone-four/enhanced (commit beefb7b).
    - From that folder: pip install -r requirements.txt, then python -m pytest tests -q, then python dashboard.py. No live Mongo is required for that path.
- Narrative: CS-499_5-2_Narrative_Databases_Amauri_Hopewell.

**Code Review:**

- The code review video is the Module Two submission, CS499 2-2 Code Review Amauri Hopewell. It is posted at https://youtu.be/ZKbE1ZThCFY.

**References:**

Hopewell, A. (2026, September 6). CS-499 Module One assignment: ePortfolio enhancement plan [Unpublished manuscript]. Southern New Hampshire University.

Southern New Hampshire University. (n.d.). CS 499 final project guidelines and rubric.

***Acknowledgment of generative AI use***

I used Grok 4.7 (xAI) to help organize this self-assessment against the Module Seven prompt, to improve my phrasing, and to format the Word file. I revised the status sentences on October 18, 2026, after the milestone grades came back without a request for more code.